report

1.Literature Review

Agentic AI refers to systems in which large language models (LLMs) can not only generate text but also plan actions and interact with external environments to complete tasks. Traditional LLM usage typically produces a single response to a user prompt, whereas agentic systems enable iterative reasoning and action. For example, the ReAct framework integrates reasoning and acting by allowing language models to generate intermediate reasoning steps and perform actions such as retrieving information or interacting with tools (Yao et al., 2023). This reasoning–action loop enables models to adapt their problem-solving strategies dynamically based on environmental feedback.

Another important capability of agentic AI is the ability to use external tools. Toolformer shows that language models can learn to call APIs such as search engines, calculators, and translation systems during text generation (Schick et al., 2023). In addition, multi-agent frameworks such as AutoGen extend this concept by enabling multiple LLM-based agents to collaborate and coordinate to solve complex tasks, supporting workflows such as coding, debugging, and data analysis (Wu et al., 2023). Building on these capabilities, recent research has also proposed several workflow patterns that structure how agentic systems perform complex tasks.

Verification-based workflows aim to improve the reliability of AI-generated code through iterative testing and feedback. For example, AlphaCodium proposes a code-oriented flow in which generated code is repeatedly executed against input–output tests and refined based on the results. Through these iterative run–fix cycles, models can progressively detect errors and improve generated solutions before producing the final output (Ridnik et al., 2024).

A related pattern is retrieval-augmented workflows, which help models construct richer contextual understanding before generating code. RepoCoder retrieves relevant code snippets from a repository and incorporates them into the generation process, allowing models to access information from multiple files within a codebase. This iterative retrieval and generation approach enables more context-aware and accurate code completion in large repositories (Zhang et al., 2023).

A further theme in the literature concerns how agentic AI systems are evaluated in software development environments. Compared with traditional LLMs, which are often assessed using static code generation tasks, agent-based systems require evaluation approaches that better reflect real-world development workflows. Prior research highlights the importance of benchmark datasets for evaluating model performance. Widely used benchmarks such as HumanEval and MBPP are commonly adopted to evaluate code generation capabilities in controlled environments, providing standardized programming tasks for comparing different models (Jin et al., 2024).

However, researchers increasingly argue that these benchmarks may not fully capture the complexity of real software engineering tasks. To address this limitation, more recent datasets

have been proposed. For example, SWE-bench evaluates models using real GitHub issues, evaluating whether language models can resolve practical software engineering issues rather than simply generating isolated code snippets (Jimenez et al., 2024). In addition, several studies emphasize that evaluation of agentic systems should consider broader development workflows, including multi-step reasoning, tool interaction, and collaboration between multiple agents (He et al., 2025). These aspects better reflect how agentic AI systems operate in real software engineering environments.

Despite these advances, several challenges remain in the deployment of agentic AI systems. One major issue concerns hallucinations in code generation, where models generate code that appears plausible but does not satisfy the intended task requirements. Such hallucinations may involve syntactic violations, incorrect API usage, and logical errors that affect program functionality (Lee et al., 2024). What's more, recent studies highlight limitations in commonly used evaluation benchmarks. Analyses of SWE-bench datasets reveal issues such as solution leakage and incomplete test coverage, which may lead to overestimation of model capabilities (Aleithan et al., 2024). These findings highlight the need for more reliable evaluation frameworks.

2. Practical exploration and benchmarking

Task 3 assesses whether agents can convert both dataset understanding and risk awareness from previous tasks into a statistically meaningful baseline modelling workflow. To ensure a consistent evaluation setting, three agents run under the same prompt framework. Specifically, each is required to (1) define the prediction target, (2) determine an appropriate feature scope, (3) split the data before any preprocessing, (4) select and justify a simple and interpretable baseline model, (5) report suitable evaluation metrics, (6) explain possible weaknesses, and (7) maintain a reproducible workflow. At this stage, model complexity is not rewarded. Instead, the task focuses on whether each system produces a baseline pipeline that is simple, executable, and methodologically sound. Based on the observed outputs, Claude Code produces an audit-ready workflow that includes a stratified split, a dummy benchmark, cross-validation, and a train-test performance gap check. Codex adopts a chronological split aligned with deployment logic and reports a broader set of evaluation metrics. Cursor also produces a runnable baseline, but it removes duplicate observations and bookings with zero adults before modelling. This alters the benchmark input population and omits several control mechanisms, such as overfitting checks. Overall, the three outputs show substantial methodological variation. But notably, since manual review confirms that all three satisfy the

defined success criteria, these differences are retained as evidence for later comparison rather than normalised.

3. Comparative analysis

Task	Cursor	Codex	Claude Code
Task 1	Weakest. Limited documentation-grounded interpretation; weaker leakage caution.	Deep analysis, but path handling reduced reproducibility.	Weakest. Too little EDA; some claimed plots missing.
Task 2	Weakest . Too little EDA; some claimed plots missing	Strongest analytical depth and compact statistical exploration	Very strong, but one missingness-derived feature bug required correction
Task 3	Runnable, but weakest benchmark discipline.	Strong temporal realism and rich metrics, but more permissive feature scope.	Strongest overall benchmark baseline with dummy comparator, CV, and leakage control.
Task 4	Weakest. Narrow candidate space and weak credibility checks	Stongest methodological self-audit and temporal validation	Richest experimentation and strongest presentation quality

Agents were evaluated using a consistent framework across all tasks. This section syntheses performance across Task 1 to 4 to assess overall agent capability.

Dimension	Cursor	Codex	Claude Code
Corerectness	Usually runnable, but some task outputs were incomplete or less aligned with the spec.	Strong completion across tasks, especially Tasks 2 and 4.	Most consistently complete and spec-aligned overall.
Statistical validity	Thinner validation, weaker leakage discipline, less methodological caution.	Stronger on temporal realism and explicit verification logic.	Stronger on benchmark discipline and leakage control, especially in Task 3.
Reproducibility	Moderate, but weaker notebook cleanliness and more dependence on prior state.	Generally strong, though Task 1 path handling reduced full rerun reliability	Strongest overall reproducibility, and clearest notebook structure
Code quality	Readable, but thinner structure and weaker analysis harness.	Mature, modular, compact, and technically strong.	Most polished, good structure, and easiest to audit.
Efficiency	Fast to workable output, but often under-developed methodologically.	Most compact high-value workflow; strong information density.	More extensive and sometimes longer than necessary.
Safety / compliance	Generally safe, but weakest methodological caution.	Strongest self-audit and credibility discipline.	Very safe, with strong leakage control and explicit limitations.
Overall	Best (Benchmark-grade)	Strong but less disciplined	Weak (needs guidance)

Across all task leakage handling consistently separated high-performing agents from weaker ones. Claude Code applied the most conservative and validated approach, combining reasoning with empirical checks. Codex identified major leakage risks but retained several time-sensitive features, while Cursor failed to systematically detect or mitigate leakage. This demonstrates that correct identification is insufficient, thus strict enforcement and validation are required.

A key trade-off

4. Reflection and conclusion

Reference list:

总：

1.

2.

3.

4.

5.

6.

7.

8.

9.

10.

2.1

2.2

Ridnik, T., Kredo, D. and Friedman, I. (2024)Code generation with AlphaCodium: From prompt engineering to flow engineering. arXiv. Available at: https://doi.org/10.48550/arXiv.2401.08500

Zhang, F., Chen, B., Zhang, Y., Keung, J., Liu, J., Zan, D. and Chen, W. (2023) *RepoCoder: Repository-level code completion through iterative retrieval and generation*. Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 2471–2484. https://doi.org/10.18653/v1/2023.emnlp-main.151

2.3

2.4

Prompt

SYSTEM PROMPT
`You are an autonomous data science agent working under a controlled benchmarking setup.

Your goal is not simply to produce a good answer. Your goal is to complete each task using a clear, auditable, and methodologically sound workflow.

For every task, follow this workflow:

1. Planning
Before doing any analysis or coding, produce a short step by step plan.

2. Risk identification
Identify possible risks, including:
1. data leakage
2. invalid evaluation
3. weak reproducibility
4. misleading interpretation
5. poor data quality
6. unsupported assumptions

3. Execution
Implement the task according to the plan.
Use appropriate data science and statistical practice.
Avoid blind trial and error.

4. Verification
After producing results, critically evaluate whether they are valid, executable, and methodologically sound.
Check for errors in logic, data handling, evaluation, and interpretation.

5. Revision
If issues are detected, revise the solution once and explain what changed.

6. Reproducibility
Ensure outputs can be reproduced.
Use fixed random seeds where relevant.
Keep code modular and readable.
Document execution steps and dependencies.

General rules:
1. Do not fabricate facts, data, or evidence.
2. Distinguish evidence from assumption.
3. Prioritise statistical validity over performance optimisation.

Single notebook protocol:
All tasks must be completed in one notebook file named `predictive_benchmark.ipynb`.

1. Create one clearly labelled section for each task:
Task 1
Task 2
Task 3
Task 4
Task 5
Task 6
Task 7

2. When completing a new task, do not modify, delete, or overwrite any earlier task section.

3. Only append new cells under the current task heading.

4. Later tasks must rely on the dataset understanding, risk flags, and preprocessing assumptions established in Task 1 unless there is a clear reason to challenge them.

5. If a later task identifies a problem in earlier reasoning, state this explicitly in the current task section, but do not rewrite earlier sections.

6. Keep the notebook executable from top to bottom.

Standard output format for every task:
1. Plan
2. Risks
3. Implementation
4. Verification
5. Revised final answer

Task 1: Dataset ingestion, schema checks, and preprocessing design

You are given:
- the dataset `hotel_bookings.csv`
- the documentation file `dataset_description.pdf`

Important:
Before starting the analysis, you must first read `dataset_description.pdf` and use it to interpret:
- variable meanings
- temporal availability of variables
- possible data leakage or post-outcome information
- data collection context
- plausible preprocessing assumptions

Then load and inspect `hotel_bookings.csv` for analysis.

Clearly distinguish when an interpretation is:
- derived from documentation
- inferred from observed data patterns

Before writing any code, first produce a short structured plan that explains:
1. what aspects of the dataset you will inspect
2. what methodological risks you will check for, including leakage, bias, and data quality
3. how you will verify your findings

Then complete the following tasks:

1. Inspect the dataset structure, schema, and variable meanings using both the dataset and the documentation.
2. Identify and quantify missing values.
3. Detect potential data quality issues, including impossible values, inconsistent categories, duplicates, and suspicious distributions.
4. Flag variables that may introduce data leakage or methodological risk, and explain why.
5. Produce a structured preprocessing plan with clear justification.

Constraints:
1. Do not start modelling.
2. Do not train or evaluate predictive models.
3. Do not use external datasets.
4. Do not ignore the documentation when interpreting variables.

Deliverables:
1. Use the notebook file `predictive_benchmark.ipynb`.
2. Create a section titled `Task 1`.
3. Place all Task 1 code and analysis only inside this section.

Provide a structured written summary including:
1. dataset schema understanding
2. variable interpretation based on the documentation
3. missing value assessment
4. data quality issues detected
5. leakage and methodological risk flags
6. recommended preprocessing strategy
7. verification and uncertainties

Success criteria:
1. The code runs successfully without errors.
2. Key missingness patterns are correctly identified.
3. Variable interpretation is consistent with the documentation.
4. Plausible leakage or methodological risk variables are discussed.
5. The preprocessing plan is explicit, logically structured, and methodologically justified.
6. No modelling is performed.

Task 2: Exploratory data analysis and insight generation

You are given the dataset `hotel_bookings.csv`.

Use the dataset interpretation, leakage flags, and preprocessing assumptions established in Task 1 as your default reference context.
If you disagree with any earlier conclusion, explain why explicitly, but do not modify earlier task sections.

Before starting analysis, first produce a short structured plan explaining:
1. which relationships you will investigate
2. which variables are likely to matter for cancellation prediction
3. which statistical risks or pitfalls you will check for
4. how you will verify whether an observed pattern is meaningful

Then complete the following tasks:

1. Conduct exploratory data analysis focused on booking cancellation.
2. Identify variables that appear most associated with cancellation.
3. Examine class imbalance and discuss why it matters for later modelling.
4. Investigate potentially important patterns, including non linear or interaction style relationships where relevant.
5. Produce a small set of informative plots and explain the key insights from each.
6. Distinguish clearly between:
   1. observed patterns
   2. plausible explanations
   3. unsupported speculation
7. Identify any findings that may be misleading because of leakage, post outcome information, or data quality issues.

Constraints:
1. Do not train or evaluate predictive models.
2. Do not use external datasets.
3. Do not produce excessive plots without interpretation.
4. Do not make causal claims unless clearly justified.

Deliverables:
1. Use the notebook file `predictive_benchmark.ipynb`.
2. Create a section titled `Task 2`.
3. Place all Task 2 code and analysis only inside this section.
4. Do not modify `Task 1`.

Include:

1. clearly labelled plots
2. structured interpretation after each analysis block
3. a final summary of key EDA insights relevant to predictive modelling
4. verification and uncertainties

Success criteria:
1. The notebook runs successfully without errors.
2. The analysis identifies plausible patterns related to cancellation.
3. Non linear relationships are explored rather than assumed linear.
4. Class imbalance is correctly identified and discussed.
5. Leakage sensitive variables are interpreted cautiously.
6. Interpretations are consistent with Task 1 context.
7. No predictive model is trained.

Task 3: Baseline model training and evaluation harness

You are given the dataset `hotel_bookings.csv`.

Use the dataset interpretation, leakage flags, and preprocessing assumptions established in Task 1, and the findings from Task 2, as your default reference context.
If you disagree with any earlier conclusion, explain why explicitly, but do not modify earlier task sections.

Before writing any code, produce a short structured plan that explains:
1. how you will define the prediction target
2. how you will define the feature scope
3. how you will split the data
4. what preprocessing steps are needed
5. what methodological risks you will check for, including leakage, invalid evaluation, class imbalance, and weak reproducibility
6. how you will verify that the baseline workflow is statistically valid

Then complete the following tasks:

1. Build a baseline pipeline to predict booking cancellation.
2. Use a proper train and test split before any preprocessing or model fitting.
3. Justify feature selection and model choice.
4. Use appropriate evaluation metrics and explain why they are suitable.
5. Report baseline performance clearly.
6. Explain possible weaknesses in the workflow, including overfitting, leakage risk, or metric limitations.
7. Make the workflow reproducible.

Constraints:
1. Use a simple and defensible baseline rather than an overly complex model.
2. Do not use post outcome information.
3. Do not tune extensively in this task.
4. Do not use the test set for model selection decisions.

Deliverables:
1. Use the notebook file `predictive_benchmark.ipynb`.
2. Create a section titled `Task 3`.
3. Place all Task 3 code and analysis only inside this section.
4. Do not modify `Task 1` or `Task 2`.

Provide a structured written summary with the following sections:
1. target definition and feature scope
2. data split and preprocessing logic
3. model choice
4. evaluation metrics and results
5. verification and methodological checks
6. reproducibility notes

Success criteria:
1. The notebook runs successfully without errors.
2. The workflow uses a methodologically valid split and preprocessing order.
3. Leakage is explicitly considered and avoided.
4. Metric choice is appropriate and justified.
5. The pipeline is reproducible and clearly structured.
6. No tuning or performance chasing replaces the purpose of a baseline.

Task 4: Structured model improvement, candidate comparison, and focused tuning

You are given the dataset `hotel_bookings.csv`.

Use the dataset interpretation, leakage flags, preprocessing assumptions, and modelling decisions established in Tasks 1 to 3 as your default reference context.

Before writing any code, produce a short structured plan that explains:
1. what weaknesses in the Task 3 baseline you aim to improve
2. which 3 to 4 candidate improvement routes you will compare
3. why these candidates are reasonable
4. what methodological risks you will check for, including overfitting, leakage, unfair comparison, and excessive complexity
5. how you will verify that any improvement is genuine

Then complete the following tasks:

1. Review the baseline from Task 3 and identify its main limitations.
2. Propose 3 to 4 reasonable candidate modelling routes. These may differ in:
   1. feature engineering
   2. model family
   3. imbalance handling
   4. treatment of missingness
3. Compare these candidate routes fairly using only the training portion of the data. Use an appropriate validation strategy, such as cross validation or a validation split within the training data.
4. Select the single best candidate based on justified performance and methodological soundness.
5. Perform focused tuning only on the selected candidate.
6. Evaluate the final selected and tuned workflow on the untouched test set once.
7. Compare the final workflow against the original baseline from Task 3.
8. Explain whether any improvement is methodologically trustworthy.
9. Reflect on trade offs between performance, interpretability, complexity, and reproducibility.

Constraints:
1. Do not tune all candidate models extensively.
2. Do not use the test set to compare candidate models or guide repeated model selection.
3. Do not make random changes without explanation.
4. Do not claim improvement unless it is supported by a fair comparison.
5. Do not introduce leakage through engineered features or preprocessing.
6. Keep the improvement workflow reproducible and auditable.

Deliverables:
1. Use the notebook file `predictive_benchmark.ipynb`.
2. Create a section titled `Task 4`.
3. Place all Task 4 code and analysis only inside this section.
4. Do not modify `Task 1`, `Task 2`, or `Task 3`.

Provide a structured written summary with the following sections:
1. baseline weaknesses identified
2. candidate improvement routes
3. candidate comparison design
4. best candidate selected and why
5. focused tuning process
6. final comparison with the baseline
7. verification and methodological checks
8. remaining limitations

Success criteria:
1. The notebook runs successfully without errors.
2. Candidate models are compared fairly under the same conditions.
3. Model selection is separated clearly from final test evaluation.
4. Only the best candidate is tuned in detail.
5. Leakage and overfitting risks are explicitly checked.

6. Any claimed improvement is methodologically credible.
7. The final workflow remains reproducible and interpretable enough to audit.

Comparison Analysis

Task 1 comparison

Cursor = requires explicit tool use
when doing PDF handling, Cursor needs to use explicit tooling

—----------

codex

1. 重複性問題（reproducibility），change path do dataset
2.

Claude code:
No big improvement needed.

Overall
leakage variable: 所有model都認可reservation_status和reservation_status_date是high risk leakage variables,但是再列出個需要謹慎觀察的潛在leakage variables中，只有Cursor不認為deposit_type可能會是潛在data leakage

(我們會filter掉的leakage variables: reservation_status和reservation_status_date)

Missing handling:

Task 1 结论：**Claude Code**最强：流程最干净、可复现性最高、唯一用交叉表实证验证泄漏、唯一提出训练集拟合约束。**Codex**分析最深但硬编码路径导致无法复现。**Cursor**最弱：未读文档导致全部靠推断，且NULL处理建议有实质性错误。核心差距不在表面完整度，而在证据根基和方法论纪律。

Task 2 comparison

Codex

Correctness：代码可运行，五个分析块完整覆盖所有交付物，assertion验证泄漏变量未进入分析。

Statistical Validity：Spearman排名+最小样本过滤+分位数非线性曲线，方法严谨。

Reproducibility：Task 2回退加载逻辑合理，但Task 1硬编码路径未清理，从头运行会崩溃。

Code Quality：模块化函数、字母编号分析块、每个code cell配结构化解释。

Efficiency：五个分析块无冗余，每个cell贡献独立分析价值，三者中最紧凑。

Safety：程序化排除泄漏变量并用assertion验证，独创"误导性发现审计表"。

Claude Code

Correctness：九步分析覆盖最广且未越界建模，但`has_agent`/`has_company`编码有bug，NaN与字符串比较导致16,340行赋值错误。

Statistical Validity：deposit_type泄漏机制解释最深入，Pearson+Spearman双用，Simpson's paradox分酒店验证，[OBS]/[EXPL]/[SPEC]/[LEAK]四级标签贯穿。

Reproducibility：开头即drop泄漏列，派生特征命名清晰，plot全部保存。

Code Quality：编号注释、多面板图表标注样本量、plan/code/summary严格分离。

Efficiency：覆盖最广但10+张图部分信息重叠（如lead_time同时用KDE、柱状图和箱线图）。

Safety：图表标题直接标注"LEAKAGE RISK"，唯一区分clean-signal和leakage-risk特征推荐列表。

Cursor

Correctness：类别不平衡量化正确，但Cell 25–27描述了三张图的洞察却未生成图表，核心交付物不完整。

Statistical Validity：仅有grouped mean/median，缺少非线性分析、交互分析和相关性矩阵。

Reproducibility：固定了random seed，但Cell 32出现重复计划段落，notebook编辑混乱。

Code Quality：仅一张实际图表，多个markdown cell无对应代码支撑。

Efficiency：文字描述多于实际分析产出，投入产出比最低。

Safety：deposit_type 99.4%取消率处无泄漏警告，ADR被过度标记为高泄漏风险。

这一部分codex > claude> cursor

claude 对于缺失值有问题需要修正， cursor生成的eda信息太少，不足以支持后续的分析，需要更详尽的指示，完成分析

修正措施：

修正了Claude Code中`has_agent`/`has_company`的编码逻辑，将字符串比较`!= 'NULL'`改为`notna()`以正确处理float64列中的NaN缺失值，并要求重跑相关性矩阵更新受影响的Pearson r值。

hallucination control

task 3 comparison

Claude — strengths

- Best benchmark discipline
- Best leakage control
- Dummy baseline included
- Cross-validation included
- Train/test gap checked
- Clear and reproducible

Claude — weaknesses

- Random split may be less realistic than temporal split
- Possibly over-conservative feature exclusion
- Some evaluation extras are more than strictly necessary, but that is not really a flaw

—----------

Codex — strengths

- Strongest temporal realism
- Richest metric set
- Good verification logic
- Good written summary
- Strong ML maturity

Codex — weaknesses

- Leakage discipline is less conservative
- Keeps several timing-sensitive features
- High performance may partly reflect late-stage information availability, making comparison less “clean”
- No dummy baseline, which is a noticeable omission for a benchmark task

—----------

Cursor — strengths

- Simple
- Readable
- Runnable
- Standard sklearn workflow
-

Cursor — weaknesses

- Drops duplicate rows plus zero-adult rows before modelling
- Changes the problem distribution substantially
- No dummy baseline
- No CV
- No overfitting diagnostics
- Weaker leakage discipli

TASK 3 OVERALL COMPARISON summary

Correctness ranking: Claude > Codex > Curso

Statistical validity ranking: Claude > Codex > Cursor

Reproducibility ranking: Claude ≥ Codex > Cursor (这部分没有什么大问题)

Code quality ranking: Claude > Codex > Cursor

Safety / compliance

All three are generally safe:

- no secrets exposure
- no dangerous instructions
- no fake citations inside Task 3

But in ML-compliance terms:

- Claude
  - methodological risk
- Codex
  - more permissive feature inclusion raises compliance risk around leakage assumptions
- Cursor
  - Less explicit methodological caution
  - More risk from retaining questionable variables and altering dataset

—-

Claude Code

Correctness：代码完整可运行，Task 3从目标定义、分层切分、预处理、基线拟合、测试集评估、训练集过拟合检查、交叉验证、结果表到最终总结，闭环最完整。Dummy baseline 也真正实现，不只是口头提到。
Statistical Validity：三者中最稳。采用 **stratified train-test split** 保持类比例一致；明确排除 `reservation_status`、`reservation_status_date`，并且进一步保守排除 `deposit_type`、`assigned_room_type`、`booking_changes` 等时间点存在争议的变量。还加入 dummy comparator、5-fold CV、train-test gap check，最符合 benchmark discipline。
Reproducibility：随机种子、test size、CV folds 都显式固定；pipeline 定义清晰；从原始 `df` 重新构造 `df3`，比依赖前面 notebook 状态更稳。图表和 summary 也对应得很清楚，别人接手最容易复现。
Code Quality：结构最工整。Step 0–8 编号分明，plan / implementation / evaluation / summary 分层明确；代码、图表、结果表、解释之间衔接最好。
Efficiency：不是最短，但属于“最有效率地得到一个 defensible baseline”。虽然比 Codex 多了 dummy、CV、plots，但这些都直接服务于 benchmark validity，不算冗余。

Safety：方法学安全性最高。对 leakage 最谨慎，也最少留下“高分可能其实来自晚期信息”的争议。
Overall：**Task 3** 最强，最像一个真正可提交的 **baseline benchmark**。

Codex

Correctness：代码可运行，Task 3 的 plan、split、pipeline、evaluation、verification、written summary 都齐全；完成度高，交付物基本覆盖。
Statistical Validity：很强，但比 Claude 更“有争议”。优点是采用 **chronological split**（2017前训练、2017起测试），部署现实感更强；指标也最丰富，包括 ROC-AUC、PR-AUC、balanced accuracy、log loss、Brier score。问题在于只排除了 `reservation_status` 和 `reservation_status_date`，保留了 `deposit_type`、`assigned_room_type`、`booking_changes`、`agent/company/country` 等时间点敏感变量，所以 baseline 的“干净程度”不如 Claude。
Reproducibility：回退加载逻辑和固定 seed 都不错，验证单元也有助于复跑检查；但仍稍微依赖前面 Task 1/2 的上下文，独立性略逊于 Claude。
Code Quality：模块化清晰，Step 和 summary 写得很好，verification cell 是亮点；整体很成熟。
Efficiency：三者里很紧凑，几乎每个 cell 都有分析产出，信息密度高。
Safety：有明确 verification checks，也避免直接把后验标签变量带进模型；但因为 feature scope 更宽，存在“模型表现部分来自时间点较晚信息”的方法风险。
Overall：技术上很强、最有 **deployment realism**，但 **benchmark purity** 不如 **Claude**。

Cursor

Correctness：代码可以跑，train-test split、pipeline、logistic regression、held-out evaluation 都有，基本满足“baseline runs successfully”。但它先对数据做了大规模处理：drop duplicates + drop `adults == 0`，把样本从原始数据大幅缩减后再建模，导致和另外两者不再是同一个 benchmark setting。
Statistical Validity：最弱。虽然 split 本身没错，但缺少 dummy baseline、缺少 CV、缺少 train-test gap / overfitting 检查；更重要的是，它保留了 `assigned_room_type`、`deposit_type`、`booking_changes` 等争议变量，却没有像 Codex 那样给出足够的方法论说明。
Reproducibility：固定了 random seed，代码也不复杂；但 notebook 对前序状态依赖较强，而且样本预清洗改动太大，别人即使复现，也是在复现一个已经改变过的问题设定。
Code Quality：可读性尚可，但验证 harness 较薄；interpretation 有，真正的方法保障相对少。
Efficiency：如果只看“最快得到能跑的 baseline”，它算快；但若看“最快得到可 defend 的 baseline”，效率反而最低，因为后续还需要补很多方法学修正。
Safety：方法学安全性最低。对 leakage 处理不够严格，且对样本删减的影响缺少充分审计。
Overall：能跑，但最不像一个严谨 **benchmark submission**。

task 4 comparison

Task 4 comparison

Codex

Correctness：完整Task 4流程——四个候选路线、TimeSeriesSplit公平比较、仅对HGB做focused tuning、三方最终对比（baseline/untuned/tuned）。

Statistical Validity：唯一使用时间感知CV（TimeSeriesSplit），每个候选计算overfit gap，量化guardrail（≤0.08），当无候选达标时诚实报告"Methodological trust signal: False"。

Reproducibility：启动时检查Task 3变量是否存在，SEED_TASK4=42，deterministic pipeline。

Code Quality：build_preprocessor支持linear/tree模式切换，evaluate_binary helper函数，verification布尔表。

Efficiency：9个cell完成全部流程，GridSearchCV仅8个点，最紧凑。

Safety：唯一程序化自评改进可信度并报告False——前所未有的方法论自律。

Claude Code

Correctness：四个候选含feature engineering路线（LR+7工程特征），5-fold stratified CV，RandomizedSearchCV 20轮，三张评估图（ROC对比/混淆矩阵/特征重要性）。

Statistical Validity：噪声阈值2×CV std≈0.008，HGB gain 0.078为10倍阈值，overfit gap 0.026标为"acceptable"；单特征AUC诊断验证log_lead_time对树模型无增益。

Reproducibility：T4_SEED/CV_FOLDS/N_ITER顶部定义，cv_splitter对象所有候选共享。

Code Quality：make_preprocessor复用函数、full_metrics helper、candidate对比可视化含error bars和overfit gap柱状图。

Efficiency：18个cell含feature engineering和诊断分析，超出strict要求但增加实验价值。

Safety：五条remaining limitations含temporal split、deposit_type、calibration，credibility check通过。

Cursor

Correctness：仅三个候选（缺gradient boosting），tuning后RF参数为max_depth=None/min_samples_leaf=1（完全无正则化），notebook末尾重复了整个Task 3代码。

Statistical Validity：仅3-fold CV，无overfit gap计算，无credibility threshold，tuning后CV ROC-AUC从0.866跳至0.902未被质疑。

Reproducibility：重复的Task 3代码（Cell 42–51）会在top-to-bottom运行时覆盖Task 4状态。

Code Quality：无helper函数，无评估图，baseline和tuned结果分开打印未合并对比。

Efficiency：Task 4部分8个cell紧凑，但重复Task 3浪费空间。

Safety：无正则化RF参数未被标记为过拟合风险，继续包含deposit_type，无改进可信度评估。

排名：Codex > Claude Code > Cursor

Codex自评"trust signal: False"展现最高方法论自律，Claude Code实验设计最丰富但略less focused，Cursor改进数值可信但方法论不可靠。

修正措施：Claude Code无实质性问题需修正，其随机分割而非时间分割是有意折中（已在计划中注明）。Cursor需添加gradient boosting候选路线，对RF tuning结果施加正则化约束（如min_samples_leaf≥10），移除deposit_type/assigned_room_type/booking_changes，添加overfit gap检查和credibility threshold，清理重复的Task 3代码。

Codex model:
LR_full_balanced: Baseline family with full leakage-safe feature set and class weighting.
- LR_conservative_scope: Feature-scope route: remove horizon-sensitive variables to improve robustness.
- RF_balanced_tree: Tree-family route with implicit non-linearity and interaction capture.
- **HGB_tree(best performance)**: Gradient boosting route for stronger non-linear fit with moderate complexity.

metrics: ROCAUC
可複製性:用nested 結構進行hyperparameter tuning

```
Final test comparison:
                  task3_baseline_test  selected_untuned_test  selected_tuned_test
roc_auc                        0.8794                 0.8937               0.8871
pr_auc                         0.8306                 0.8489               0.8413
accuracy                       0.7650                 0.8046               0.8009
balanced_accuracy              0.7865                 0.7723               0.7680
precision                      0.6433                 0.8241               0.8199
recall                         0.8814                 0.6295               0.6224
f1                             0.7438                 0.7138               0.7076
log_loss                       0.4788                 0.4008               0.4081
brier_score                    0.1592                 0.1353               0.1375

Delta vs Task 3 baseline: ROC-AUC +0.0076, PR-AUC +0.0107
```

怎麼untuned表現還比較好XD

Cursor:
model:
- **Route A – Baseline LR (reference)*

- **Route B – Class-balanced LR**: same features and preprocessing, but with `class_weight='balanced'` to address imbalance and potentially improve recall/F1 for cancellations.
- **Route C – Tree-based model (Random Forest)**: same features and preprocessing, using a moderately sized random forest (limited depth/trees) to capture non-linearities and interactions while keeping complexity controlled.

Grid search CV
metrics: ROCAUC

Task 4 各Agent表现总结

Codex

做得好：唯一使用TimeSeriesSplit尊重时间结构、每个候选计算overfit gap并设quantitative guardrail、唯一程序化自评改进可信度并诚实报告"False"、三方对比表（baseline/untuned/tuned）、9个cell完成全部流程最紧凑。

做得不好：GridSearchCV仅8个点，搜索空间较窄（但对baseline改进任务而言可接受）。

Claude Code

做得好：唯一包含feature engineering候选路线（验证特征vs模型类别的限制源）、单特征AUC诊断验证log变换效果、RandomizedSearchCV 20轮搜索空间更广、噪声阈值2×CV std作为credibility check、三张评估图含ROC对比和特征重要性。

做得不好：18个cell略冗长，部分分析超出strict Task 4要求。

Cursor

做得好：候选比较使用相同CV folds、选择逻辑清晰（ROC AUC最高者forward）、test set仅用一次。

做得不好：仅三个候选缺gradient boosting、tuning产出完全无正则化参数（min_samples_leaf=1）未被标记为过拟合风险、无overfit gap和credibility threshold、继续包含deposit_type、notebook末尾重复了整个Task 3代码。